Active Transaction Graphs: A Formal Framework for Transactional Interactive Systems

Alexander Vityaz
Corezoid Inc., Dnipro, Ukraine
alexander.vityaz@gmail.com
corezoid.com · simulator.company
ORCID: 0009-0006-0489-7881
https://doi.org/10.5281/zenodo.20747873
NotebookLM

Abstract

We introduce *Active Transaction Graphs* (ATGs), a formal framework for interactive systems in which persistent state, mediated interaction, execution traceability, and accounting consequences must be modeled simultaneously. The framework is based on five core commitments: participating entities are modeled as actors, actors may be recursively structured, formally relevant interaction is transactional, graph edges are first-class computational entities, and observational semantics is determined by the triple $(result, trace, ledger)$.

We distinguish explicitly between the minimal formal core of the framework, derived results, architectural principles, and conjectural extensions. Within the formal layer, we prove a noise projection theorem, show that finite Petri nets embed into ATGs, and show that CHAM-style systems arise as quotients of ATGs under suitable erasures. We further prove that explicit edge mediation is semantically non-eliminable in ledger-sensitive mediated systems. A worked example based on invoice approval and payment illustrates the formalism.

The framework can be extended upward with a signal layer, whose projection onto the transactional layer explains the classification of noise as semantically irrelevant interaction.

The resulting framework is intended not as a replacement for classical models of computation but as a semantic envelope for transactional interactive systems, including workflow engines, enterprise platforms, and mixed human–AI operational environments.

1 Introduction

Classical models of computation typically emphasize one dominant aspect of system behavior: functional transformation, concurrency, rewriting, or communication. Turing machines privilege function computation [1]; lambda calculus privileges symbolic reduction [2]; Petri nets privilege concurrency and resource flow [3]; the Chemical Abstract Machine (CHAM) privileges reaction-style execution [5]. These models are powerful, but large interactive systems often require several semantic layers at once:

- persistent local state,
- structured interaction,
- execution traceability,
- accounting consequences,
- explicit mediation and control,
- hierarchical regulation.

Such requirements are common in enterprise platforms, workflow engines, payment systems, human–AI operational environments, and mixed socio-technical organizations [7, 8, 9].

This paper introduces *Active Transaction Graphs* (ATGs), a formal framework intended to capture these layers within a single ontology. The central idea is simple: participating entities are modeled as actors, but interaction is modeled not primarily by messages; instead it is modeled by *transaction templates*, whose executions produce state change, trace effects, and ledger effects. Moreover, graph edges are not passive relations: they are themselves actors that mediate, constrain, and account for transactional flow.

The contribution of the paper is fourfold:

1. it defines a minimal formal core for active transaction graphs;
2. it introduces an observational semantics based on the triple $(result, trace, ledger)$;
3. it proves several elementary results, including a noise projection theorem, a Petri-net embedding theorem, a CHAM quotient theorem, and a non-eliminability proposition for edge mediation in ledger-sensitive settings;
4. it separates formal results from architectural principles and conjectural extensions.

A key design choice of the paper is methodological stratification. We distinguish: (i) axioms required for the minimal formal core, (ii) theorems derivable within that core, (iii) architectural principles suggested by the framework, and (iv) conjectural extensions intended as a research program. This separation is essential because several intuitively attractive ideas in the broader ATG picture—such as variational computation, hierarchical closure, and energetic actor geometry— are promising but not yet mature enough to be treated as foundational.

The paper is structured accordingly. Sections 2–7 contain the formal core, the signal extension, derived results, relations to prior models, and a worked example. Sections 8–10 present higher-level principles and open conjectures. Section 11 concludes.

2 Minimal Formal Core

2.1 Actors

Definition 2.1 (Actor)**.** An *actor* is any entity possessing:

1. an internal state,
2. one or more interaction interfaces,
3. a transition capability allowing state change in response to admissible interactions.

Axiom 2.2 (Universal Actor Axiom)**.** Any entity satisfying Definition 2.1 may be modeled as an actor.

The framework is intentionally substrate-independent. An actor may correspond to a software service, a human participant, an AI model, an AI agent, a workflow stage, a budgetary control unit, a team, or an organization.

Definition 2.3 (Recursive Actor)**.** An actor is *recursive* if it may itself be represented as an internal graph of actors.

Axiom 2.4 (Recursive Actor Principle)**.** Any actor may be modeled both as a node in a graph and as a graph of subordinate actors.

This permits multiscale description. For example, an organization may be represented as an actor at one level and as a graph of humans, services, departments, and policies at another.

2.2 Active Transaction Graphs and Edge-Actors

Definition 2.5 (Edge-Actor)**.** An *edge-actor* is an actor whose primary role is to mediate, transform, constrain, or account for interactions among actors.

Axiom 2.6 (Edge Actor Principle)**.** Graph edges are first-class computational entities.

This means that routing, retry logic, auditing, billing, rate limiting, access control, and execution policy need not be treated as external "middleware". They may instead be represented directly as first-class computational entities in the graph.

Definition 2.7 (Active Transaction Graph)**.** An *active transaction graph* is a pair

$$G = (A, E),$$

where A is a set of actors and E is a set of edge-actors.

Definition 2.8 (Configuration)**.** A *configuration* of an active transaction graph $G = (A, E)$ is an assignment of internal states to all actors and edge-actors, together with any additional control data required to determine template applicability and execution effects.

2.3 Transaction Templates and Step Semantics

Definition 2.9 (Transaction Template)**.** A *transaction template* over $G = (A, E)$ is a tuple

$$\vartheta = (e, \Sigma, \pi, x, \rho),$$

where

- $e \in E$ is the mediating edge-actor,
- $\Sigma \subseteq A$ is a finite actor support set,
- π is an interaction protocol or template type,
- x is an input payload,
- ρ is metadata including any relevant tracing, accounting, retry, SLA, policy, or execution-control information.

Definition 2.10 (Template Universe)**.** For an active transaction graph G, let

$$\Theta(G)$$

denote the set of all transaction templates over G.

Remark 2.11. A template does not contain the observable result. The result is produced only after execution in a configuration. This separates transaction specification from transaction outcome.

Axiom 2.12 (Transaction Primacy)**.** Formally relevant interaction is transactional.

The point of this axiom is not that every signal must immediately become a transaction. Rather, the axiom states that the semantically relevant unit of interaction is a transaction template together with its execution semantics, rather than a bare message.

Definition 2.13 (Finite Support Semantics)**.** Each edge-actor e may inspect and update the states of a finite actor neighborhood. Accordingly, every transaction template carries a finite support set $\Sigma \subseteq A$. In ordinary point-to-point cases $|\Sigma| = 2$, but more general finite-support mediated interactions are permitted.

Remark 2.14. Definition 2.13 is crucial for embeddings such as Petri nets: the framework is therefore better understood as a mediated finite-support interaction semantics rather than as a purely binary graph calculus.

Definition 2.15 (Local Applicability)**.** Let c be a configuration of G, and let

$$\vartheta = (e, \Sigma, \pi, x, \rho) \in \Theta(G).$$

We say that ϑ is *locally applicable* in configuration c if:

1. every actor in Σ exists in c,
2. the mediating edge-actor e exists in c,
3. the protocol π is admitted by e over the support Σ,
4. all state-dependent preconditions required by e, Σ, and x are satisfied.

Remark 2.16. A future refinement may make preconditions explicit via a predicate family such as

$$\mathrm{pre}_{e,\pi}(c, \Sigma, x) \in \{0, 1\}.$$

For the present formal layer, the abstract applicability condition is sufficient.

Definition 2.17 (Execution Step)**.** An *execution step* of G is a judgment of the form

$$c \xRightarrow{(\mathrm{id},\vartheta)\,/\,y,\delta_{\mathrm{Tr}},\delta_{\mathrm{Led}}} c',$$

where:

- $\vartheta \in \Theta(G)$ is locally applicable in c,
- id is an occurrence identifier,
- c' is the resulting configuration,
- y is the observable step result,
- δ_{Tr} is the trace contribution of the step,
- δ_{Led} is the ledger contribution of the step.

Definition 2.18 (Execution)**.** An *execution* of G is a finite or infinite sequence of execution steps

$$c_0 \xRightarrow{(\mathrm{id}_1,\vartheta_1)/y_1,\delta_{\mathrm{Tr},1},\delta_{\mathrm{Led},1}} c_1 \xRightarrow{(\mathrm{id}_2,\vartheta_2)/y_2,\delta_{\mathrm{Tr},2},\delta_{\mathrm{Led},2}} c_2 \cdots$$

such that each ϑ_i is locally applicable in c_{i-1} and the identifiers id_i are pairwise distinct. The class of all such executions is denoted $\mathrm{Exec}(G)$.

Remark 2.19 (Freshness Policy)**.** Identifiers are required to be unique only *per occurrence in an execution*. Thus the same template may be executed arbitrarily many times, provided each occurrence receives a fresh identifier.

Definition 2.20 (Finite Execution)**.** A *finite execution* is a finite execution prefix of the form

$$e = c_0 \xRightarrow{(\mathrm{id}_1,\vartheta_1)/y_1,\delta_{\mathrm{Tr},1},\delta_{\mathrm{Led},1}} c_1 \cdots \xRightarrow{(\mathrm{id}_n,\vartheta_n)/y_n,\delta_{\mathrm{Tr},n},\delta_{\mathrm{Led},n}} c_n.$$

The class of finite executions of G is denoted $\mathrm{Exec}_{\mathrm{fin}}(G)$.

Definition 2.21 (Identifier Set of a Finite Execution)**.** For a finite execution e as in Definition 2.20, define

$$\mathrm{Ids}(e) = \{\mathrm{id}_1, \ldots, \mathrm{id}_n\}.$$

Definition 2.22 (Template Sequence of a Finite Execution)**.** For a finite execution e as in Definition 2.20, define

$$\mathrm{Desc}(e) = \vartheta_1 \cdots \vartheta_n \in \Theta(G)^*.$$

That is, $\mathrm{Desc}(e)$ records the sequence of templates while erasing occurrence identifiers and step outcomes.

Definition 2.23 (Composable Finite Executions)**.** Let

$$e_1 : c_0 \Rightarrow \cdots \Rightarrow c_n, \qquad e_2 : d_0 \Rightarrow \cdots \Rightarrow d_m$$

be finite executions of G. We say that e_1 and e_2 are *composable* if:

1. $c_n = d_0$,
2. $\mathrm{Ids}(e_1) \cap \mathrm{Ids}(e_2) = \varnothing$.

Their *concatenation* $e_1 \cdot e_2$ is the finite execution obtained by concatenating the two step sequences.

2.4 Observation Interface and Observational Equivalence

Definition 2.24 (Observation Interface)**.** Let X be any system equipped with a class $\mathrm{Exec}_{\mathrm{fin}}(X)$ of finite runs. An *observation interface* for X consists of three maps

$$\mathrm{Res}_X : \mathrm{Exec}_{\mathrm{fin}}(X) \to \mathcal{R}_X, \qquad \mathrm{Tr}_X : \mathrm{Exec}_{\mathrm{fin}}(X) \to \mathcal{T}_X, \qquad \mathrm{Led}_X : \mathrm{Exec}_{\mathrm{fin}}(X) \to \mathcal{L}_X,$$

where $\mathcal{R}_X, \mathcal{T}_X, \mathcal{L}_X$ are, respectively, the result, trace, and ledger codomains associated with X.

Definition 2.25 (Observation Triple)**.** Given an observation interface on X, the *observation triple* of a finite run $e \in \mathrm{Exec}_{\mathrm{fin}}(X)$ is

$$\mathrm{Obs}_X(e) = \big(\mathrm{Res}_X(e), \mathrm{Tr}_X(e), \mathrm{Led}_X(e)\big).$$

Axiom 2.26 (Observational Triple Axiom)**.** Every ATG considered in this paper is equipped with an observation interface, and observational semantics is determined by the triple

$$(\mathit{result}, \mathit{trace}, \mathit{ledger}).$$

Definition 2.27 (Observational Equivalence)**.** Two finite executions $e_1, e_2 \in \mathrm{Exec}_{\mathrm{fin}}(G)$ are *observationally equivalent*, written

$$e_1 \sim_{\mathrm{obs}} e_2,$$

if

$$\mathrm{Obs}_G(e_1) = \mathrm{Obs}_G(e_2).$$

Remark 2.28 (Compatibility with Step Data)**.** In typical instantiations, Tr_G and Led_G are obtained by aggregating the step-level deltas $\delta_{\mathrm{Tr},i}$ and $\delta_{\mathrm{Led},i}$, while Res_G is extracted from step results, final configurations, or both. The present framework leaves this aggregation discipline application-dependent.

Definition 2.29 (Passive-Edge Representation)**.** A *passive-edge representation* of an ATG G is a related interaction system $G^\flat$ equipped with:

1. a class $\mathrm{Exec}_{\mathrm{fin}}(G^\flat)$ of finite runs,
2. an observation interface

$$\mathrm{Res}_{G^\flat} : \mathrm{Exec}_{\mathrm{fin}}(G^\flat) \to \mathcal{R}_G,$$
$$\mathrm{Tr}_{G^\flat} : \mathrm{Exec}_{\mathrm{fin}}(G^\flat) \to \mathcal{T}_G,$$
$$\mathrm{Led}_{G^\flat} : \mathrm{Exec}_{\mathrm{fin}}(G^\flat) \to \mathcal{L}_G,$$

obtained by replacing one or more edge-actors of G with passive relations while preserving the same support incidences and intended interaction signatures.

Proposition 2.30 (Edge Elimination Can Change Observations Across Systems)**.** *There exist an ATG G, a passive-edge representation $G^\flat$, and finite runs $e \in \mathrm{Exec}_{\mathrm{fin}}(G)$, $e^\flat \in \mathrm{Exec}_{\mathrm{fin}}(G^\flat)$, such that*

$$\mathrm{Obs}_G(e) \neq \mathrm{Obs}_{G^\flat}(e^\flat).$$

Proof. Consider an ATG with actors A and B, and an edge-actor E mediating a template

$$\vartheta = (E, \{A, B\}, \pi, x, \rho).$$

Assume that every successful execution step through E produces a ledger increment corresponding to a routing fee.

Let e be a one-step finite execution of G containing an occurrence of ϑ. Then $\mathrm{Led}_G(e)$ contains the routing fee.

Now form a passive-edge representation $G^\flat$ by replacing E with a passive relation between A and B, and let $e^\flat$ be the corresponding one-step run connecting the same support actors under the same protocol and payload. Because the passive edge is not a computational entity, it cannot itself generate the routing-fee contribution.

Therefore either the routing fee is absent from $\mathrm{Led}_{G^\flat}(e^\flat)$, or some new computational mediator has been introduced elsewhere, in which case the system is no longer passive-edge only. In the passive-edge case,

$$\mathrm{Obs}_G(e) \neq \mathrm{Obs}_{G^\flat}(e^\flat)$$

because the ledger components differ. □

3 Extended Signal Layer

The minimal formal core of ATGs begins at the transactional layer. However, many practical systems involve signals or messages that do not automatically rise to transactional status. To distinguish semantically relevant interaction from merely potential interaction, we introduce an extended signal layer.

Definition 3.1 (Signal)**.** A *signal* is an interaction event, message, observation, or external perturbation that may or may not acquire transactional consequence.

Definition 3.2 (Signal History)**.** Let S be a set of signals. A *finite signal history* over S is a finite sequence

$$h = s_1 s_2 \cdots s_n \in S^*.$$

We write $\mathrm{Hist}(S) = S^*$ for the set of all finite signal histories.

Definition 3.3 (Extended Signal–Transaction System)**.** An *extended signal–transaction system* is a triple

$$G^+ = (G, S, R),$$

where:

- G is an underlying active transaction graph,
- S is a set of signals,
- $R : S \to \Theta(G)^*$ is a *signal realization map*, sending each signal to a finite sequence of transaction templates over G.

Remark 3.4. The map R may encode direct realization, preparation, validation, authorization, or explicit suppression, provided these are represented at the transactional layer by appropriate template types or protocols.

Definition 3.5 (Realized Template Sequence)**.** The realization map extends uniquely to signal histories by concatenation:

$$R^*(s_1 \cdots s_n) = R(s_1) \cdots R(s_n) \in \Theta(G)^*.$$

Definition 3.6 (Transactional Projection of a Signal History)**.** Let $G^+ = (G, S, R)$ be an extended signal–transaction system. For a finite signal history $h \in \mathrm{Hist}(S)$, the *transactional projection* of h is the set

$$\Pi_T(h) \subseteq \mathrm{Exec}_{\mathrm{fin}}(G)$$

defined by

$$\Pi_T(h) = \{e \in \mathrm{Exec}_{\mathrm{fin}}(G) \mid \mathrm{Desc}(e) = R^*(h)\}.$$

Remark 3.7. The original ATG formalism may be regarded as operating after signal histories have been projected to finite transactional executions. Thus the signal layer extends ATG upward without changing the minimal transactional core.

Remark 3.8. Transaction primacy should be read semantically rather than chronologically: it states that ATG semantics is defined on the projected transactional layer, even if the larger system contains additional signal-level events.

Remark 3.9. If a specific initial configuration c_0 is fixed, one may refine $\Pi_T(h)$ to the subset of projected executions beginning at c_0. The present paper leaves the initial-state discipline application-dependent.

Definition 3.10 (Transactional Consequence at the Signal Layer)**.** A signal $s \in S$ has *transactional consequence* if its realization sequence is nonempty:

$$R(s) \neq \varepsilon.$$

Definition 3.11 (Signal Noise)**.** A signal $s \in S$ is *noise* if it has no transactional consequence, i.e.

$$R(s) = \varepsilon.$$

The projection semantics for signal noise used here parallels the noise-suppression factorization for minimal good regulators in [10].

Theorem 3.12 (Noise Projection Theorem)**.** *Let $G^+ = (G, S, R)$ be an extended signal–transaction system, and let $s \in S$ be a signal such that $R(s) = \varepsilon$. Then for all finite signal histories $u, v \in \mathrm{Hist}(S)$,*

$$\Pi_T(usv) = \Pi_T(uv).$$

Hence s is invisible at the transactional layer.

Proof. By Definition 3.5,

$$R^*(usv) = R^*(u)\, R(s)\, R^*(v).$$

If $R(s) = \varepsilon$, then

$$R^*(usv) = R^*(u)\, \varepsilon\, R^*(v) = R^*(uv).$$

By Definition 3.6, the transactional projection depends only on the realized template sequence. Hence

$$\Pi_T(usv) = \Pi_T(uv).$$

□

Corollary 3.13 (Noise Invariance of Observational Semantics)**.** *Let $s \in S$ be noise in $G^+ = (G, S, R)$. Then for all finite signal histories $u, v \in \mathrm{Hist}(S)$,*

$$\{\mathrm{Obs}_G(e) \mid e \in \Pi_T(usv)\} = \{\mathrm{Obs}_G(e) \mid e \in \Pi_T(uv)\}.$$

Proof. By Theorem 3.12, $\Pi_T(usv) = \Pi_T(uv)$. Applying Definition 2.25 to the same set of finite executions yields the same set of observation triples. □

4 Noise and Transactional Relevance

At the ATG level, noise is understood as the image under projection of signal-level events with no transactional consequence.

Principle 4.1 (Message-to-Transaction Principle)**.** A signal or message without transactional consequence is noise.

Remark 4.2. The principle is justified in two steps: first at the signal layer by Theorem 3.12, and then at the ATG layer by Corollary 3.13.

5 Ledger Semantics

Definition 5.1 (Ledger Functional)**.** A *ledger functional* for G is a mapping

$$\Lambda : \mathrm{Exec}_{\mathrm{fin}}(G) \to \mathcal{L},$$

from finite executions to accounting states in some ledger space $\mathcal{L}$.

Definition 5.2 (Ledger-Preserving Equivalence)**.** Two finite executions $e_1, e_2 \in \mathrm{Exec}_{\mathrm{fin}}(G)$ are *ledger-preservingly equivalent*, written

$$e_1 \sim_L e_2,$$

if

$$\Lambda(e_1) = \Lambda(e_2).$$

Definition 5.3 (Compositional Ledger Functional)**.** A ledger functional is *compositional* if there exists a binary operation

$$\odot : \mathcal{L} \times \mathcal{L} \to \mathcal{L}$$

such that for any two composable finite executions e_1, e_2,

$$\Lambda(e_1 \cdot e_2) = \Lambda(e_1) \odot \Lambda(e_2).$$

Proposition 5.4. *If the ledger functional is compositional, and*

$$e_1 \sim_L e_1', \qquad e_2 \sim_L e_2',$$

with e_1, e_2 composable and e_1', e_2' composable, then

$$e_1 \cdot e_2 \sim_L e_1' \cdot e_2'.$$

Proof. From

$$e_1 \sim_L e_1', \qquad e_2 \sim_L e_2'$$

we obtain

$$\Lambda(e_1) = \Lambda(e_1'), \qquad \Lambda(e_2) = \Lambda(e_2').$$

Since the pairs are composable and Λ is compositional,

$$\Lambda(e_1 \cdot e_2) = \Lambda(e_1) \odot \Lambda(e_2)$$

and

$$\Lambda(e_1' \cdot e_2') = \Lambda(e_1') \odot \Lambda(e_2').$$

Substituting equal terms yields

$$\Lambda(e_1 \cdot e_2) = \Lambda(e_1' \cdot e_2').$$

Hence

$$e_1 \cdot e_2 \sim_L e_1' \cdot e_2'.$$

□

Remark 5.5. The framework does not require a globally additive conservation law in full generality. Some systems may conserve balances, others may conserve quotas, budgets, or obligations, and still others may only support weaker ledger invariants. A stronger conservation principle is better treated as an application-specific or conjectural layer.

6 Relations and Comparison with Existing Models

The ATG framework is best understood not as a competitor to classical models but as a semantic envelope for a class of systems in which interaction, mediation, traceability, and accounting must be modeled simultaneously.

Turing machines [1] and the lambda calculus [2] are canonical models of function-oriented computation. Their strength lies in expressivity for algorithmic transformation, but they do not natively distinguish: (i) mediated interaction, (ii) execution trace as a semantic object, or (iii) accounting consequences of interaction. ATGs do not attempt to replace these models in their native domain. Rather, they target systems in which execution is not adequately captured by function evaluation alone.

Petri nets [3] provide an elegant model of concurrency, resource flow, and synchronization. Their core semantics is token-based and transition-centered. ATGs preserve the ability to model local state and transition constraints, while extending the semantic vocabulary in three directions: transitions become first-class edge-mediators, execution traces can be treated semantically rather than operationally, and ledger consequences can be included natively.

CHAM [5] models computation as reactions over a structured solution. This is close in spirit to the present framework, especially in its emphasis on dynamic configurations over fixed programs. The difference is that ATGs enrich the reaction model with: identifiable transaction templates, explicit mediation by edge-actors, trace semantics, and ledger semantics.

In classical actor systems [4], actors communicate through messages. ATGs agree with the actor tradition on local state and decentralized interaction, but differ in taking *transaction* rather than *message* as the primary formally relevant unit. ATGs also differ by making mediation explicit: what is often treated as middleware, infrastructure, or runtime policy in practice can be represented as edge-actors in the graph itself.

Process calculi, especially π-calculus [6], provide powerful tools for channel-based mobility and process interaction. ATGs are less fine-grained in the syntactic theory of name mobility, but are more explicit about: persistent actor identity, edge-level mediation, and result–trace–ledger observational semantics.

To summarize these differences, Table 1 compares ATGs with several classical models across the semantic dimensions most relevant to the present framework.

Model	Persistent local state	Explicit interaction semantics	First-class mediation	Trace-aware semantics	Ledger / accounting semantics
Turing machine	Limited / global tape	No	No	No	No
Lambda calculus	Indirect	No	No	No	No
Petri nets	Yes (markings)	Yes (token flow)	No	Weak	No
CHAM	Yes (configurations)	Yes (reactions)	No	Weak	No
Hewitt actors	Yes	Yes (messages)	Limited / externalized	Weak / impl.-dep.	No
π-calculus	Process-local	Yes (channels)	Indirect	Weak / derived	No
Active Transaction Graphs	Yes	Yes (transactions)	Yes (edge-actors)	Yes	Yes

Table 1: Semantic comparison of ATGs with selected classical models.

The table should be read with caution. For example, Petri nets and CHAM both admit rich extensions, and actor systems may be augmented with tracing or accounting in practice. The point is not that such enrichments are impossible elsewhere, but that in ATGs they are part of the intended semantic vocabulary rather than external instrumentation.

6.1 CHAM as a Quotient of ATG

Definition 6.1 (CHAM Quotient of an ATG)**.** Let G be an active transaction graph. Its *CHAM quotient* is obtained by:

1. erasing occurrence identifiers,
2. erasing ledger structure,
3. quotienting trace detail by internal mediation,
4. collapsing edge-actors into reaction rules over configurations.

Theorem 6.2 (CHAM Quotient Theorem)**.** *The CHAM quotient of an active transaction graph admits a CHAM-style reaction representation.*

Proof. Let G be an ATG. Construct a CHAM-style representation as follows.

Associate to each actor state a molecule-like term. Let the global configuration be the multiset, or more generally the structured collection, of all such actor-state terms.

For each transaction template together with its mediating edge behavior, after erasing occurrence identifiers, ledger structure, and internal mediation detail, introduce a reaction rule transforming the corresponding source-state pattern into the corresponding destination-state pattern.

Because the quotient construction removes identifier-sensitive, ledger-sensitive, and mediation-sensitive distinctions, only configuration-transforming behavior remains. Thus the quotient system can be represented as a CHAM-style reaction system over configurations. □

Remark 6.3. Theorem 6.2 converts the informal chemical analogy into a precise reduction statement: CHAM appears not merely as a vague inspiration but as a quotient of an ATG under specific erasures.

6.2 Petri Nets as a Special Case of ATG

Definition 6.4 (Finite Petri Net)**.** A finite Petri net is a tuple

$$N = (P, T, F, M_0),$$

where:

- P is a finite set of places,
- T is a finite set of transitions,
- $F \subseteq (P \times T) \cup (T \times P)$ is the flow relation,
- $M_0 : P \to \mathbb{N}$ is the initial marking.

Theorem 6.5 (Petri Net Embedding Theorem)**.** *Any finite Petri net can be represented as a special case of an active transaction graph.*

Proof. Let $N = (P, T, F, M_0)$ be a finite Petri net.

Construct an ATG $G = (A, E)$ as follows.

Actors. For each place $p \in P$, introduce an actor A_p. Its state contains the current token count:

$$\sigma(A_p) = M(p) \in \mathbb{N}.$$

Edge-actors. For each Petri transition $t \in T$, introduce an edge-actor E_t.

Finite-support mediation. Let $\Sigma_t \subseteq A$ be the set of all actors corresponding to places in the preset and postset of t. The edge-actor E_t inspects and updates exactly this finite support set.

Templates. A firing of t is represented by the transaction template

$$\vartheta_t = (E_t, \Sigma_t, \pi_t, x_t, \rho_t).$$

Occurrences. Repeated firings of the same Petri transition are represented by fresh execution occurrences of the same template ϑ_t, each with its own fresh identifier.

Step semantics. The precondition for ϑ_t is precisely the Petri-net enabledness condition: the input-place actors in Σ_t must carry sufficient tokens. When locally applicable, the corresponding execution step removes tokens from input places and adds tokens to output places exactly as in the marking update of t.

Thus:

- places become actors carrying local token-count state;
- transitions become edge-actors;
- markings become global assignments of local actor state;
- firing becomes a finite-support mediated transaction step.

If one ignores ledger enrichment and quotients trace semantics down to firing order, the induced execution semantics coincides with the standard sequential firing-sequence semantics of the original finite Petri net. Therefore the Petri net is represented as a special case of an ATG. □

Remark 6.6. Theorem 6.5 shows that ATGs subsume finite Petri nets once mediated finite-support interactions are admitted. This is why the framework should not be read as merely binary graph semantics.

7 Worked Example: Invoice, Approval, and Payment

We now illustrate the framework with a simple enterprise-native process: invoice creation, approval, and payment.

7.1 Operational Description

Consider a process in which:

1. an invoice is created;
2. the invoice is submitted for approval;
3. an approver either approves or rejects it;
4. if approved, the payment is executed;
5. the payment is posted to a ledger.

The example is intentionally small, but it exhibits all three components of the observation triple: result, trace, and ledger.

7.2 Actors

We introduce the following actors:

- A_I: **Invoice actor**, whose state records invoice identity, amount, status, and approval flag;
- A_U: **Approver actor**, whose state records approval authority and approval decisions;
- A_P: **Payment actor**, whose state records payment execution status;
- A_L: **Ledger actor**, whose state records posted accounting entries.

For example, the invoice actor may carry a state of the form

$$\sigma(A_I) = (\text{invoice\_id}, \text{amount}, \text{status}, \text{approved}),$$

with status values such as `draft`, `submitted`, `approved`, `paid`, `rejected`.

7.3 Edge-Actors

We introduce the following edge-actors:

- E_{IA}: **Approval submission edge-actor**, which mediates invoice submission to the approver;
- E_{AP}: **Payment authorization edge-actor**, which allows payment execution only if approval conditions are met;
- E_{PL}: **Ledger posting edge-actor**, which creates ledger-relevant postings after successful payment.

These edge-actors may enforce policies such as:

- invoice completeness checks;
- approver authorization checks;
- payment idempotency;
- ledger posting format and validation.

7.4 Transaction Templates

For readability we write the three main mediated interactions informally as

$$A_I \xrightarrow{E_{IA}} A_U, \qquad A_U \xrightarrow{E_{AP}} A_P, \qquad A_P \xrightarrow{E_{PL}} A_L.$$

Formally, they are templates of the form

$$\vartheta_1 = (E_{IA}, \{A_I, A_U\}, \pi_{\text{sub}}, x_1, \rho_1),$$

$$\vartheta_2 = (E_{AP}, \{A_U, A_P\}, \pi_{\text{auth}}, x_2, \rho_2),$$

$$\vartheta_3 = (E_{PL}, \{A_P, A_L\}, \pi_{\text{post}}, x_3, \rho_3).$$

7.5 Execution Outcomes

Suppose the invoice is approved and paid successfully. Then a corresponding finite execution e yields:

Result

$$\text{Res}_G(e) = \texttt{invoice paid}.$$

Trace A relevant trace may be represented as

$$\text{Tr}_G(e) = \texttt{create} \to \texttt{submit} \to \texttt{approve} \to \texttt{pay} \to \texttt{post}.$$

Ledger The ledger projection includes a posted payment entry, for example:

$$\text{Led}_G(e) = \texttt{debit expense, credit cash}.$$

Thus the semantics of the process is not exhausted by the final status alone. A system that returns `paid` without the required trace or without a ledger posting is not observationally equivalent to the execution above.

7.6 Noise Example

Consider an informal reminder message such as "please approve this soon" sent outside the formal system.

If the signal realization map sends this message to the empty sequence, then by the signal-layer results of Section 3 it is noise relative to the chosen observational semantics.

7.7 Why Edge-Actors Matter

In this example, edge-actors are not decorative. They carry essential semantics:

- E_{IA} may reject malformed submissions;
- E_{AP} may enforce the rule that payment cannot proceed without approval;
- E_{PL} may guarantee that financial posting happens exactly once.

If these controls are left implicit, the formal model loses precisely the layer that matters most to enterprise systems: policy, accountability, and control over interaction itself.

Entity	Role	Relevant semantics
A_I	Invoice actor	Stores invoice identity, amount, lifecycle status
A_U	Approver actor	Stores approval authority and approval decisions
A_P	Payment actor	Stores payment execution state
A_L	Ledger actor	Stores posted accounting entries
E_{IA}	Submission edge-actor	Validates and routes invoice for approval
E_{AP}	Authorization edge-actor	Enforces payment only after approval
E_{PL}	Posting edge-actor	Creates a ledger-relevant posting exactly once

Table 2: Actors and edge-actors in the invoice–approval–payment example.

7.8 Actor and Edge Summary

8 Architectural Principles

The following statements are not taken as part of the minimal axiomatic core. They are higher-level principles naturally suggested by the framework.

Principle 8.1 (Transaction-as-Work Principle)**.** A transaction step acts as an elementary unit of work over state, structure, and ledger.

This can be summarized informally as

$$Work((\mathrm{id}, \vartheta)) = \Delta State + \Delta Structure + \Delta Ledger.$$

Principle 8.2 (Hierarchical Noise Suppression)**.** Hierarchical organization reduces semantic overload by separating raw event processing from stable aggregate structure.

Principle 8.3 (Chemical Interpretation)**.** An active transaction graph may be interpreted as a generalized reaction medium whose reactions are transaction steps.

Principle 8.4 (Graph Programming Thesis)**.** Programming may be understood as the construction, restriction, execution, and evolution of actor graphs.

Principle 8.5 (Metaprogram Organization Principle)**.** Organizations may be understood as self-modifying transactional systems that execute and alter their own interaction program.

Remark 8.6. These principles are intentionally separated from the minimal formal core. They help organize the intuition and broader meaning of the framework, but they are not required for the earlier theorems.

9 Extended Actor Structure

The minimal theory does not require extra energetic or geometric structure. Nevertheless, some applications may benefit from enriching actors with additional parameters.

Definition 9.1 (Structured Actor)**.** A *structured actor* is an actor equipped with a tuple

$$a = (\sigma, I, q, H, v),$$

where:

- σ is internal state,
- I is a typed interface system,
- $q \in \mathbb{R}_{\geq 0}$ is an activation weight,
- H is a finite set of unsatisfied interaction demands,
- $v \in \mathbb{N}$ is a connectivity bound.

Interpretation 9.2. In this enriched language:

- q may be read as charge or activation potential,
- H as holes or open demands,
- v as valency or a connectivity constraint.

Remark 9.3. This structure is treated here as an optional enrichment, not as part of the minimal formal core. This is deliberate: the enriched language is promising, but its algebraic consequences deserve separate development.

10 Conjectures and Open Problems

We conclude with several conjectural extensions. These are presented as research directions rather than established results.

Conjecture 10.1 (Variational Computation)**.** Among admissible trajectories of an active transaction graph, realized executions extremize a functional balancing: activity, structural tension, noise, and economic cost.

Remark 10.2. A precise formulation would require a rigorous notion of graph dynamics, including a nontrivial definition of $\dot{G}$. This conjecture therefore lies beyond the present formal core.

Hypothesis 10.3 (Church–Turing–Transactional)**.** Any effectively realizable interactive system whose semantics is expressible in terms of actor states, transaction templates, traces, and ledger projections can be represented as an active transaction graph up to observational equivalence.

Remark 10.4. This hypothesis is not intended to supersede the classical Church–Turing thesis. Rather, it redirects attention from function computation to transactional interactive systems.

Conjecture 10.5 (Minimal Good Regulator)**.** Hierarchically organized active transaction graphs admit closure points at which noise, cost, and regulatory complexity are jointly minimized subject to predictive adequacy.

Interpretation 10.6 (Double-Categorical Reading)**.** The framework admits a natural double-categorical reading: objects as actors, horizontal 1-cells as edge-actors, vertical 1-cells as refinements or projections, and 2-cells as coherence witnesses for transactional semantics.

Remark 10.7. This reading is mathematically suggestive but is not required for the formal results proved in the present paper. A full categorical treatment is deferred to future work.

11 Conclusion

We have introduced active transaction graphs as a formal framework for transactional interactive systems. The paper distinguishes sharply between:

- a *minimal formal core*,

- *derived theorems and propositions*,
- *architectural principles*,
- *conjectural extensions*.

Within the formal layer, the framework is built on five core commitments:

1. participating entities are modeled as actors;
2. actors may be recursively structured;
3. formally relevant interaction is transactional;
4. graph edges are first-class computational entities;
5. observational semantics is determined by the triple $(result, trace, ledger)$.

From this core, we proved:

- a noise projection theorem and its observational corollary,
- compositional closure of ledger-preserving equivalence,
- a CHAM quotient theorem,
- a Petri-net embedding theorem,
- non-eliminability of edge mediation in ledger-sensitive mediated systems.

We have also provided a worked example showing how ATGs capture result semantics, trace semantics, and ledger semantics simultaneously.

The broader claims of the framework—graph programming, hierarchical noise suppression, metaprogram organizations, variational computation, and transactional universality— remain promising but partly conjectural. Their separation from the minimal core is intentional: it allows the present work to function as a disciplined foundation rather than as an undifferentiated manifesto.

The novelty of the framework lies not merely in representing systems as graphs, which is commonplace, but in jointly insisting on: (i) transaction primacy over message primacy, (ii) first-class computational mediation at edges, and (iii) observational semantics that includes result, trace, and ledger. Taken together, these commitments define a semantic regime that is not standard in classical models, yet is natural for transactional interactive systems.

The main thesis of the paper may therefore be stated cautiously as follows: active transaction graphs provide a useful and extensible formal language for systems in which interaction itself must be modeled as a first-class computational and accountable phenomenon.

Supplementary Resource

NotebookLM resource

References

[1] A. M. Turing. On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(2):230–265, 1936.

[2] H. P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*. North-Holland, revised edition, 1984.

[3] C. A. Petri. *Kommunikation mit Automaten*. PhD thesis, University of Bonn, 1962.

[4] C. Hewitt, P. Bishop, and R. Steiger. A Universal Modular Actor Formalism for Artificial Intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence*, pages 235–245, 1973.

[5] G. Berry and G. Boudol. The Chemical Abstract Machine. *Theoretical Computer Science*, 96(1):217–248, 1992.

[6] R. Milner, J. Parrow, and D. Walker. A Calculus of Mobile Processes, I/II. *Information and Computation*, 100(1):1–77, 1992.

[7] J. Gray and A. Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992.

[8] W. M. P. van der Aalst. *Process Mining: Data Science in Action*. Springer, 2nd edition, 2016.

[9] J. Cheney, L. Chiticariu, and W.-C. Tan. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.

[10] O. Vityaz. On the Necessity of Noise Suppression for Minimal Good Regulators: Factorization Theorems and a Closure Conjecture. ResearchGate preprint, January 2026. doi:10.13140/RG.2.2.33143.07843.